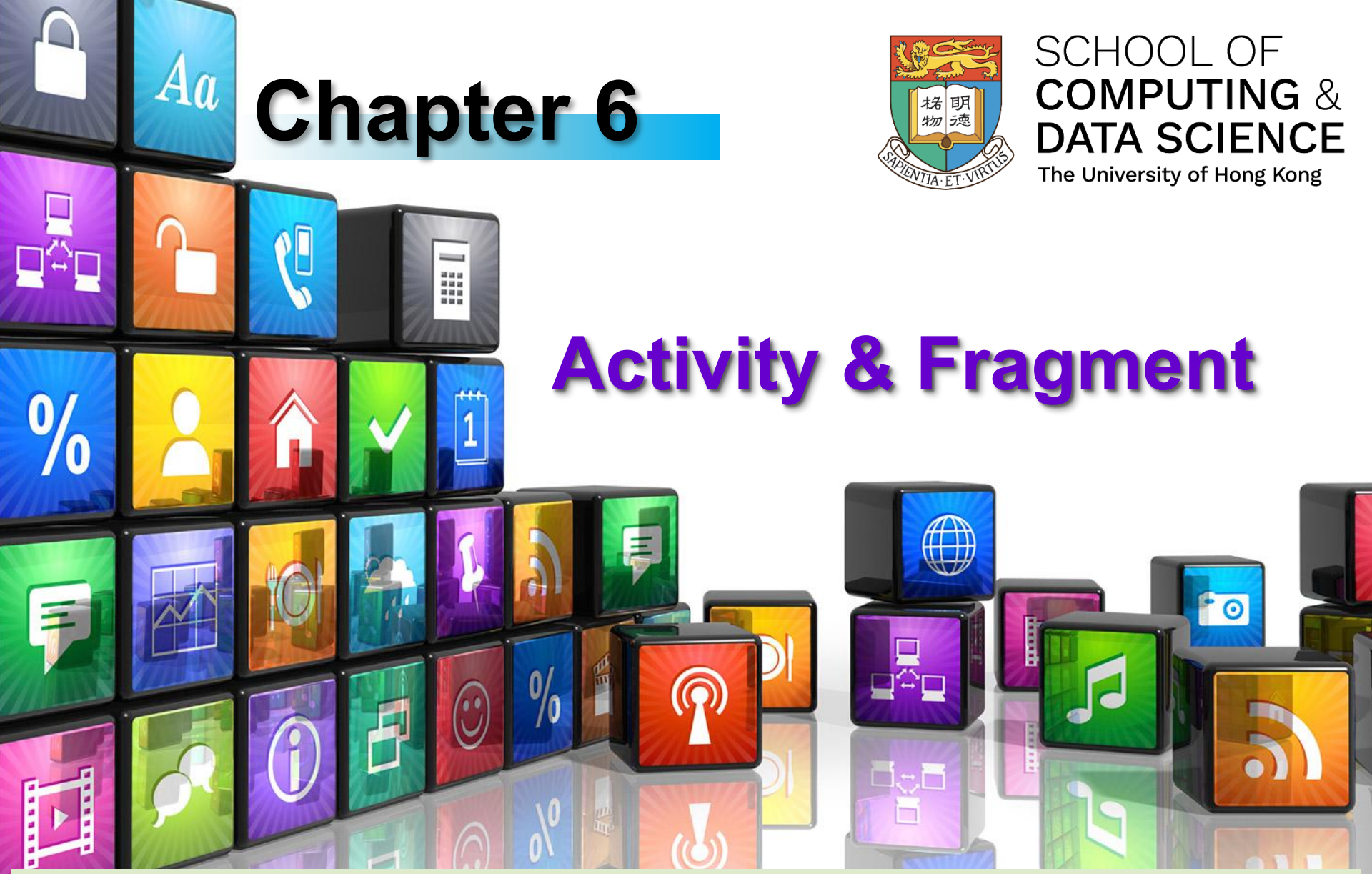
Chapter 6



SCHOOL OF COMPUTING & DATA SCIENCE
The University of Hong Kong

Activity & Fragment

2026-2027
COMP3330 Interactive Mobile Application Design and Programming
Dr. T.W. Chim (E-mail: twchim.cs@hku.hk)

Agenda

- Concept of Activity
- Intent & Filter
  - Implicit Intent Example
  - Explicit Intent Example
- Passing Extra Data
- Fragment

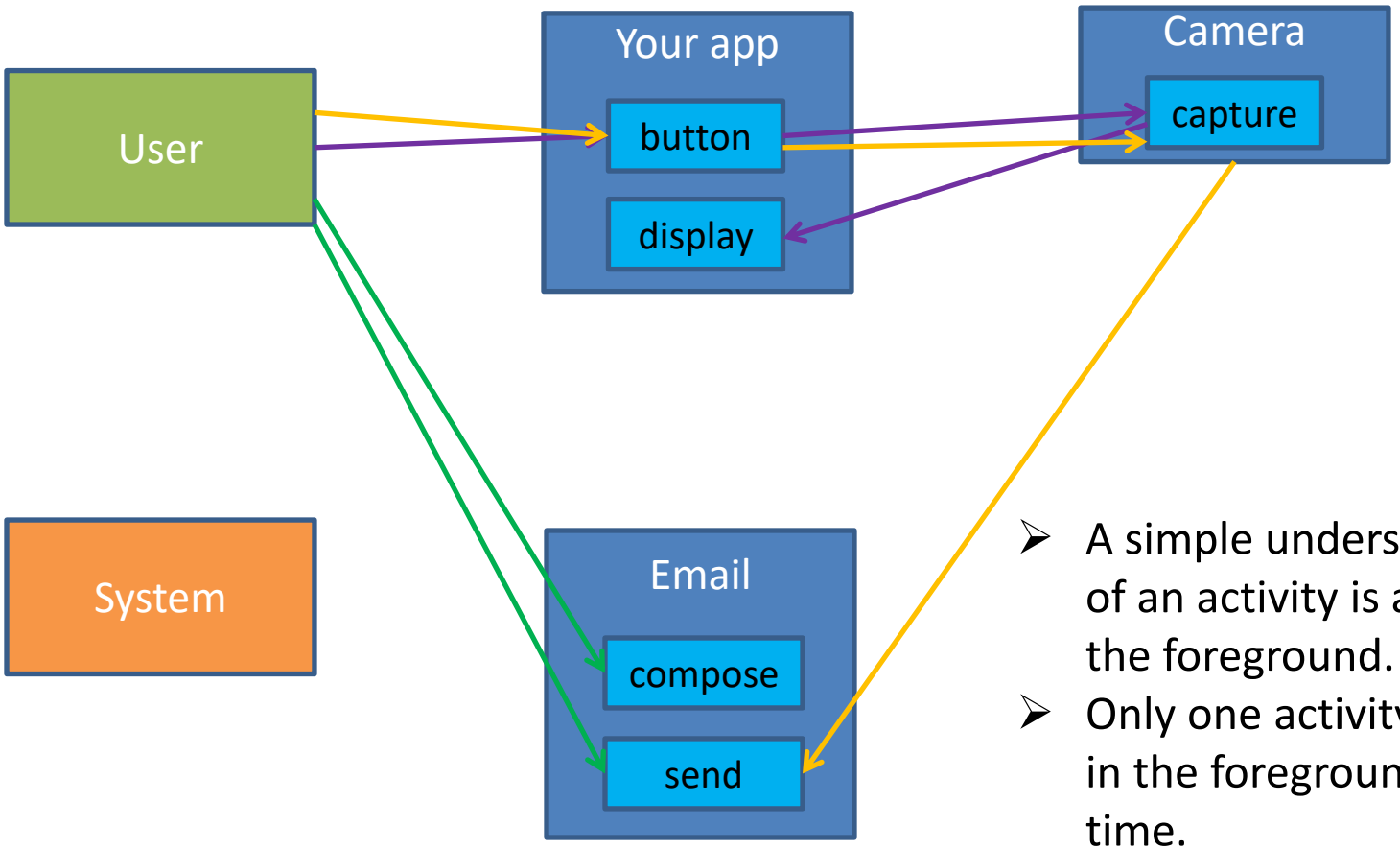
Application Components

- Activity
  - User interface / foreground
  - e.g., buttons, lists
- Service
  - No user interface / background
  - e.g., music, download
- Content provider
  - Data manager
  - e.g., files, database
- Broadcast receiver
  - Responsible for system-wide events
  - e.g., shut down, low battery, download complete

Concept of Activity



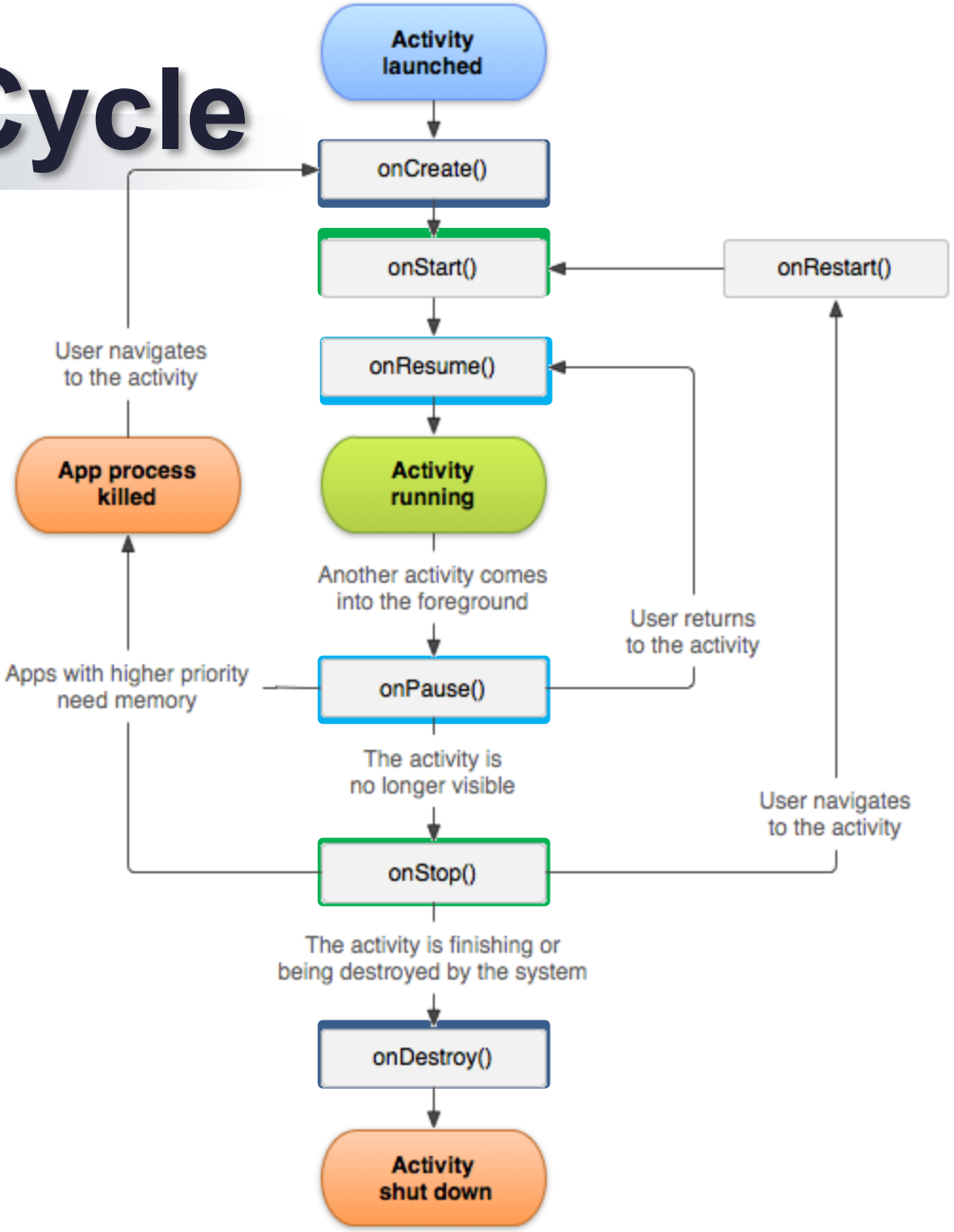
Example of Activities



- A simple understanding of an activity is a page in the foreground.
- Only one activity can be in the foreground at any time.

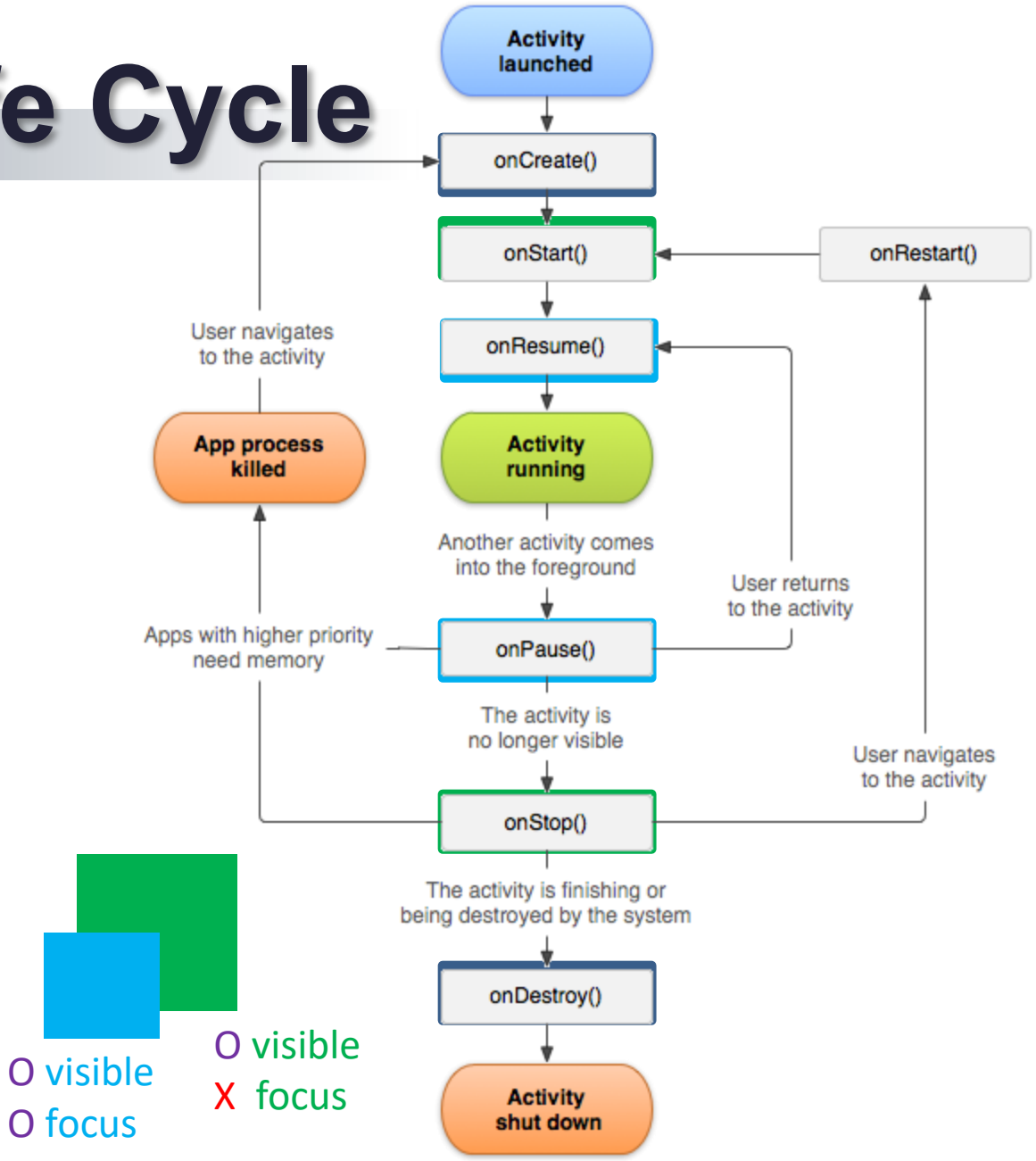
Activity Life Cycle

- Android system manages app life cycles
- Different callback functions will be invoked at different stages



Activity Life Cycle

- onCreate()
  - Enter memory
- onStart()
  - Become visible
- onResume()
  - Get focus / foreground
- onPause()
  - Lose focus / foreground
- onStop()
  - Become invisible
- onDestroy()
  - Leave memory



Note: An activity can be visible but not in focus if the phone screen can display multiple activities at the same time.

Activity Coordination (Example)

- Activity A starts Activity B:
  - 1. A's onPause() method executes.
  - 2. B's onCreate(), onStart(), and onResume() methods execute in sequence. (B now has user focus.)
  - 3. If A is no longer visible on screen, its onStop() method executes.
- So, e.g., if A needs to write to a DB before B starts, use onPause() instead of onStop().

Intent & Filter

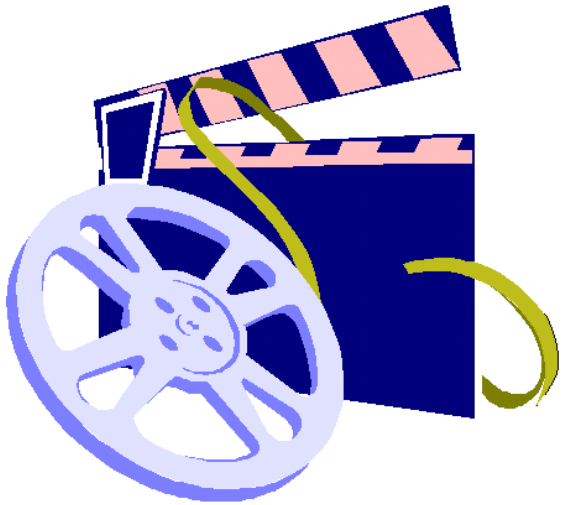


Examples

Android intents can be imagined as messages between activities (also services and receivers).

- startActivity

Intent
Action: start movie
Data: movie title



- startActivityForResult

Intent
Action: get photo

Call-back

Intent
Data: photo

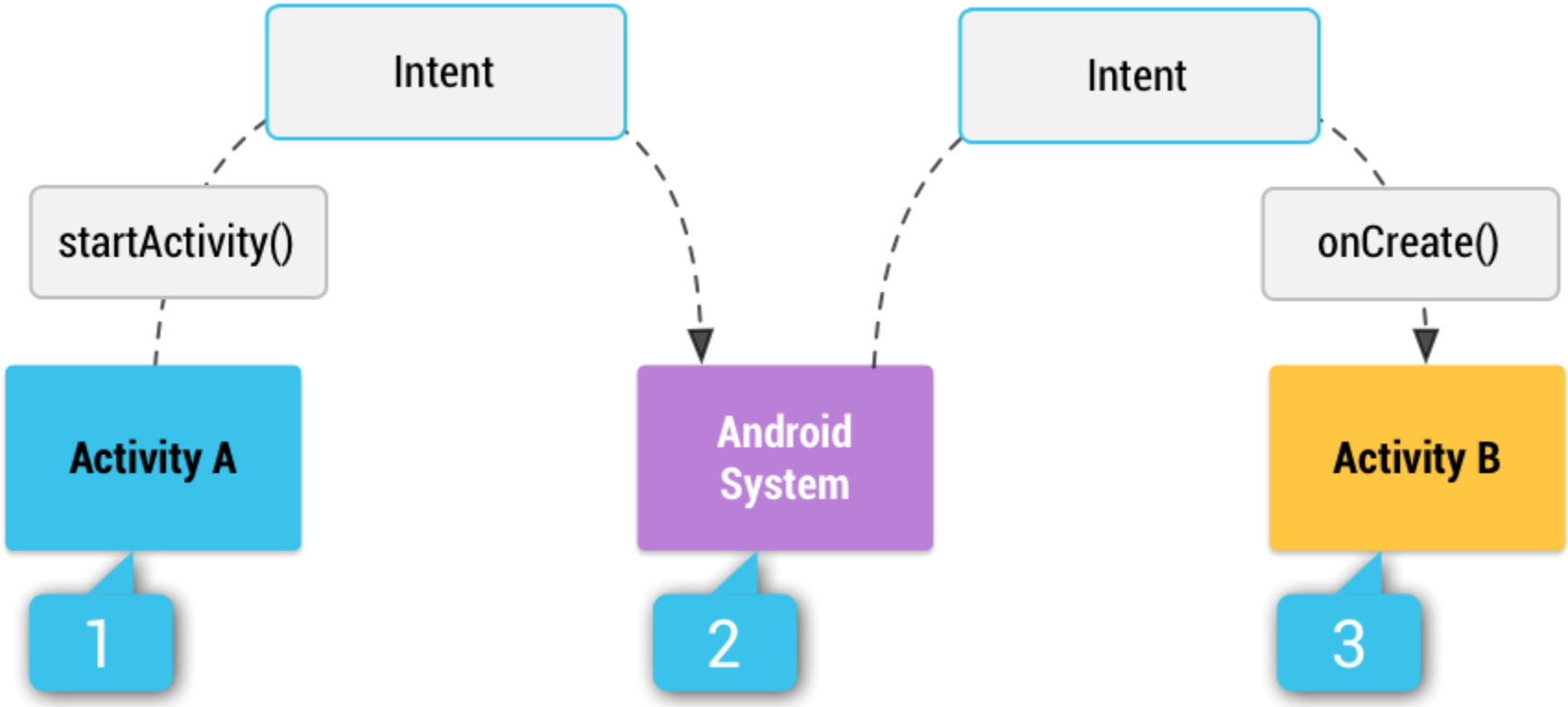
Deprecated

Camera App

Explicit vs. Implicit Intent

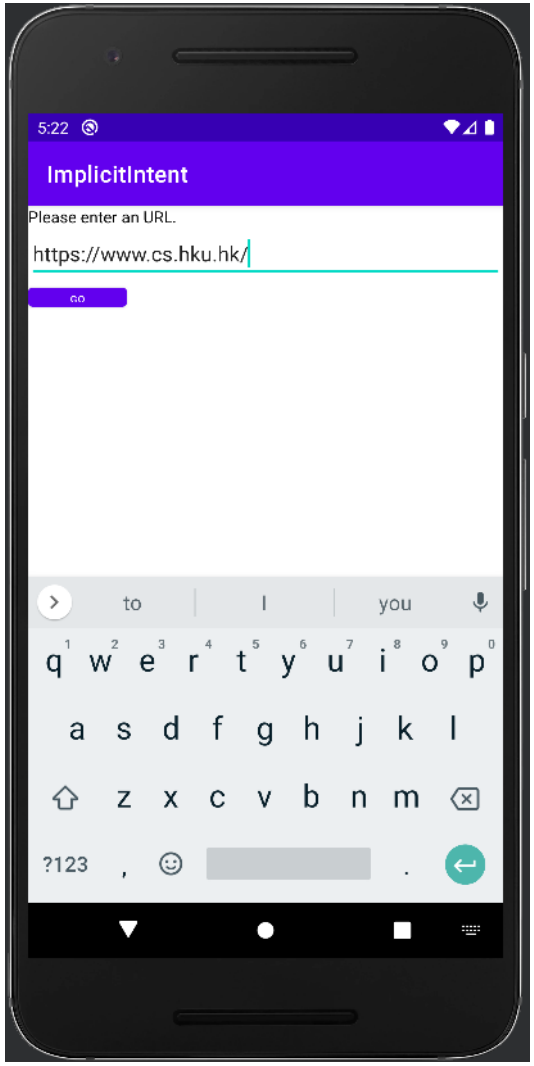
- Explicit
  - Explicit intents are used in the application itself wherein one activity can switch to another activity.
  - Java: `Intent intent = new Intent(this, FooActivity.class);`
    `startActivity(intent);`
  - Kotlin: `intent = Intent(this, FooActivity::class.java)`
    `startActivity(intent)`
- Implicit
  - Implicit intents do not directly specify the Android component which should be called, it only specifies the action to be performed. The Android component will be picked by the Android system.
  - Java: `Intent intent = new Intent(Intent.ACTION_VIEW, Uri.parse(URLStr));`
    `startActivity(intent);`
  - Kotlin: `intent = Intent(Intent.ACTION_VIEW, Uri.parse(URLStr));`
    `startActivity(intent)`

Implicit Intent Handling



Implicit Intent Example







Implicit Intent Example (AndroidManifest.xml)

```xml
<?xml version="1.0" encoding="utf-8"?>
<manifest xmlns:android="http://schemas.android.com/apk/res/android"
    xmlns:tools="http://schemas.android.com/tools"
    package="hk.hkucs.implicitintent">

    <application
        android:allowBackup="true"
        android:dataExtractionRules="@xml/data_extraction_rules"
        android:fullBackupContent="@xml/backup_rules"
        android:icon="@mipmap/ic_launcher"
        android:label="ImplicitIntent"
        android:roundIcon="@mipmap/ic_launcher_round"
        android:supportsRtl="true"
        android:theme="@style/Theme.ImplicitIntent"
        tools:targetApi="31">
        <activity
            android:name=".MainActivity"
            android:exported="true">
            <intent-filter>
                <action android:name="android.intent.action.MAIN" />

                <category android:name="android.intent.category.LAUNCHER" />
            </intent-filter>
        </activity>
    </application>

</manifest>
```

Default Manifest without any change

Implicit Intent Example (activity_main.xml)

```
<?xml version="1.0" encoding="utf-8"?>
<LinearLayout
    xmlns:android="http://schemas.android.com/apk/res/android"
    xmlns:tools="http://schemas.android.com/tools"
    android:orientation="vertical"
    android:layout_width="match_parent"
    android:layout_height="match_parent">
    <TextView android:layout_width="fill_parent"
        android:layout_height="wrap_content"
        android:textColor="#000000"
        android:text="Please enter an URL." />
    <EditText
        android:id="@+id/EditText1"
        android:layout_width="match_parent"
        android:layout_height="46dp" />
    <Button android:text="Go"
        android:id="@+id/Button01"
        android:layout_width="300px"
        android:layout_height="100px"
        android:textSize="30px" />
</LinearLayout>
```

Note: This layout just contains one EditText and one Button. You can draw the layout using interface building. You can also use other layouts you like.

Implicit Intent Example (MainActivity.kt)

```
package hk.hkucs.implicitintent

import android.content.Intent
import android.net.Uri
import androidx.appcompat.app.AppCompatActivity
import android.os.Bundle
import android.widget.Button
import android.widget.EditText

class MainActivity : AppCompatActivity() {
    override fun onCreate(savedInstanceState: Bundle?) {
        super.onCreate(savedInstanceState)
        setContentView(R.layout.activity_main)
        val go = findViewById<Button>(R.id.Button01)
        val input = findViewById<EditText>(R.id.EditText1)
        go.setOnClickListener {
            val inputString = input.text.toString()
            val myIntent = Intent(Intent.ACTION_VIEW,Uri.parse(inputString))
            startActivity(myIntent)
        }
    }
}
```

Intent.ACTION_VIEW: To display some information

Uri.parse(url): Uniform Resource Identifier (Uniform Resource Locator)



For complete example, please refer to the project “Implicit Intent”.

Intent Filter

- For implicit intents only
- Android determines the best activity or service (or set of receivers) on its own.
- It does so by comparing the contents of the Intent object to intent filters, structures associated with components that can potentially receive intents.
- Filters advertise the capabilities of a component and delimit the intents it can handle. They open the component to the possibility of receiving implicit intents of the advertised type.
- If a component does not have any intent filters, it can receive only explicit intents. A component with filters can receive both explicit and implicit intents.

Intent Filter Example

Activity definition in AndroidManifest.xml:

```
<activity android:name="hkucs.sample.Activity2">
        <intent-filter>
                <action android:name = "android.intent.action.VIEW" />
                <action android:name = "hkucs.myintentfilter" />
                <category android:name = "android.intent.category.DEFAULT" />
        </intent-filter>
</activity>
```

Intent object created in activity:

```
val i = Intent("hkucs.myintentfilter", Uri.parse("http://www.hku.hk"));
startActivity(i)
```

The system will look for intent filter named “hkucs.myintentfilter” and so “Activity2” which contains this filter will be invoked.

Intent Filter Example

The intent object contains an URL as parameter. How can the activity concerned collect it???

Intent object created in activity:

```
val i = Intent("hkucs.myintentfilter", Uri.parse("http://www.hku.hk"));
startActivity(i)
```

In Activity2, you can use **intent.data** (equivalent to **getIntent().getData()** in Java) to collect it. But please use a proper data structure to hold the value.

```
val url: Uri? = intent.data
```

You can then use the value anywhere in Activity2.

```
label.text = url.toString()
```

Intent Filter Example (AndroidManifest.xml)

```
<?xml version="1.0" encoding="utf-8"?>
<manifest xmlns:android="http://schemas.android.com/apk/res/android"
    xmlns:tools="http://schemas.android.com/tools"
    package="hk.hkucs.intentfilter">

    <application
        android:allowBackup="true"
        android:dataExtractionRules="@xml/data_extraction_rules"
        android:fullBackupContent="@xml/backup_rules"
        android:icon="@mipmap/ic_launcher"
        android:label="@string/app_name"
        android:roundIcon="@mipmap/ic_launcher_round"
        android:supportsRtl="true"
        android:theme="@style/Theme.IntentFilter"
        tools:targetApi="31">
        <activity
            android:name=".MainActivity"
            android:exported="true">
            <intent-filter>
                <action android:name="android.intent.action.MAIN" />
                <category android:name="android.intent.category.LAUNCHER" />
            </intent-filter>
        </activity>
```

Intent Filter Example (AndroidManifest.xml)

```
        <activity android:name="hk.hkucs.intentfilter.CustomActivity"
            android:exported="true">
            <intent-filter tools:ignore="AppLinkUrlError">
                <action android:name = "android.intent.action.VIEW" />
                <action android:name = "hk.hkucs.intentfilter.LAUNCH" />
                <category android:name = "android.intent.category.DEFAULT" />
                <data android:scheme = "http" />
            </intent-filter>
        </activity>
    </application>

</manifest>
```

Intent Filter Example (activity_main.xml)

```
<?xml version="1.0" encoding="utf-8"?>
<RelativeLayout
    xmlns:android="http://schemas.android.com/apk/res/android"
    xmlns:tools="http://schemas.android.com/tools"
    android:layout_width="match_parent"
    android:layout_height="match_parent">
    <Button
        android:layout_width="match_parent"
        android:layout_height="wrap_content"
        android:text="Start Browser"
        android:id="@+id/button" />
    <Button
        android:layout_width="match_parent"
        android:layout_height="wrap_content"
        android:text="Start browsing with launch action"
        android:id="@+id/button2"
        android:layout_below="@+id/button"
        android:layout_alignLeft="@+id/button"
        android:layout_alignRight="@+id/button"
        android:layout_alignStart="@+id/button"
        android:layout_alignEnd="@+id/button" />
</RelativeLayout>
```

Intent Filter Example (custom_view.xml)

```
<?xml version="1.0" encoding="utf-8"?>
<LinearLayout xmlns:android="http://schemas.android.com/apk/res/android"
    android:layout_width="match_parent"
    android:layout_height="match_parent">
    <TextView android:id="@+id/show_data"
        android:layout_width="fill_parent"
        android:layout_height="400dp"/>
</LinearLayout>
```

Intent Filter Example (MainActivity.kt)

```
package hk.hkucs.intentfilter

import android.content.Intent
import android.net.Uri
import android.os.Bundle
import android.widget.Button
import androidx.appcompat.app.AppCompatActivity

open class MainActivity : AppCompatActivity() {
    override fun onCreate(savedInstanceState: Bundle?) {
        super.onCreate(savedInstanceState)
        setContentView(R.layout.activity_main)
        val b1 = findViewById<Button>(R.id.button)
        b1.setOnClickListener {
            val i = Intent(Intent.ACTION_VIEW, Uri.parse("http://www.hku.hk"))
            startActivity(i)
        }
```

Intent Filter Example (MainActivity.kt)

```
val b2 = findViewById<Button>(R.id.button2)
        b2.setOnClickListener {
            val i = Intent("hk.hkucs.intentfilter.LAUNCH", Uri.parse("http://www.hku.hk"))
            startActivity(i)
        }
    }
}
```

Recall which activity has a similar filter action…

```
<activity android:name="hk.hkucs.intentfilter.CustomActivity"
    android:exported="true">
    <intent-filter tools:ignore="AppLinkUrlError">
        <action android:name = "android.intent.action.VIEW" />
        <action android:name = "hk.hkucs.intentfilter.LAUNCH" />
        <category android:name = "android.intent.category.DEFAULT" />
        <data android:scheme = "http" />
    </intent-filter>
</activity>
```

Intent Filter Example (CustomActivity.kt)

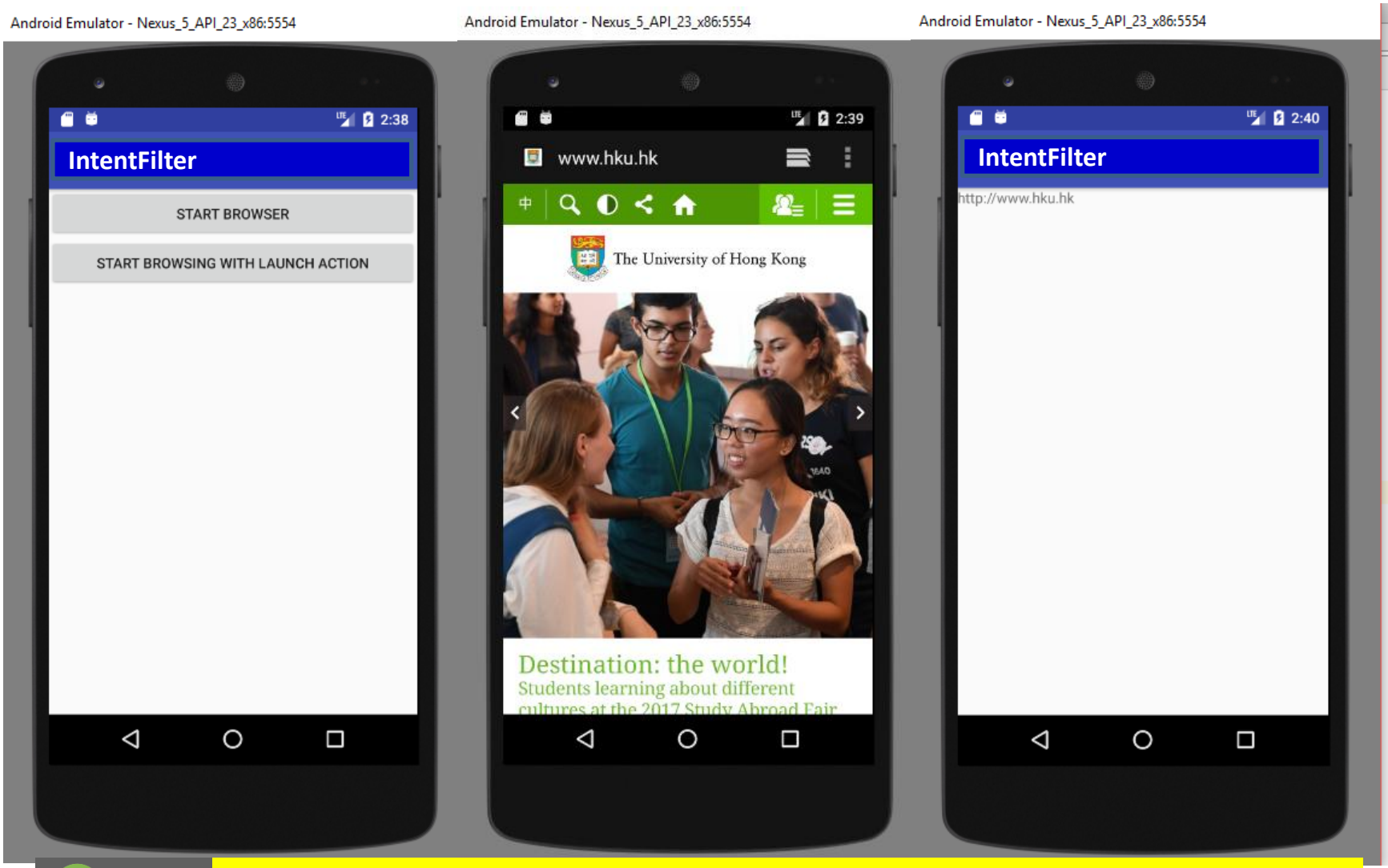
```
package hk.hkucs.intentfilter

import android.net.Uri
import android.os.Bundle
import android.widget.TextView
import androidx.appcompat.app.AppCompatActivity

class CustomActivity : AppCompatActivity() {
    override fun onCreate(savedInstanceState: Bundle?) {
        super.onCreate(savedInstanceState)
        setContentView(R.layout.custom_view)
        val label = findViewById<TextView>(R.id.show_data) as TextView
        val url: Uri? = intent.data
        label.text = url.toString()
    }
}
```

This is how to get URI data from caller…

Intent Filter Example



Android Studio

For complete example, please refer to the project "Intent Filter".

Explicit Intent

- In Android, we usually use explicit intent to do page switching (i.e., switch between different activities in the same project).
- Key steps:
  - Create a new layout for the new activity
  - Define the new activity in manifest file
  - In the calling activity (old page), jump to the new activity (new page) by creating a new Intent instance and starting it.
  - In the called activity (new page), return to the calling activity (old page) by creating another Intent instance.

Explicit Intent

Definition of activities in AndroidManifest.xml:

```
<activity android:name=".MainActivity1" >
        <intent-filter>
                <action android:name="android.intent.action.MAIN" />
                <category android:name="android.intent.category.LAUNCHER" />
        </intent-filter>
</activity>
<activity android:name=".MainActivity2">
</activity>
```

An activity having this intent filter will become the first activity (i.e., being loaded upon the app is started).

New activity

Creates an "Intent" to start another Activity. The intent needs two parameters: a context and the name of the Activity that you want to start (e.g., MainActivity2.class)

```
val myIntent = Intent(this, MainActivity2::class.java)
```

Explicit Intent Example (AndroidManifest.xml)

```
<?xml version="1.0" encoding="utf-8"?>
<manifest xmlns:android="http://schemas.android.com/apk/res/android"
    xmlns:tools="http://schemas.android.com/tools"
    package="hk.hkucs.explicitintent">
    <application
        android:allowBackup="true"
        android:dataExtractionRules="@xml/data_extraction_rules"
        android:fullBackupContent="@xml/backup_rules"
        android:icon="@mipmap/ic_launcher"
        android:label="@string/app_name"
        android:roundIcon="@mipmap/ic_launcher_round"
        android:supportsRtl="true"
        android:theme="@style/Theme.ExplicitIntent"
        tools:targetApi="31">
        <activity
            android:name=".MainActivity"
            android:exported="true">
            <intent-filter>
                <action android:name="android.intent.action.MAIN" />
                <category android:name="android.intent.category.LAUNCHER" />
            </intent-filter>
        </activity>
        <activity
            android:name=".MainActivity2">
        </activity>
    </application>
</manifest>
```

Add a new activity (a new page) here!

Explicit Intent Example (activity_main.xml)

```
<?xml version="1.0" encoding="utf-8"?>
<LinearLayout
    xmlns:android="http://schemas.android.com/apk/res/android"
    xmlns:tools="http://schemas.android.com/tools"
    android:orientation="vertical"
    android:layout_width="match_parent"
    android:layout_height="match_parent">
    <TextView android:layout_width="fill_parent"
        android:layout_height="wrap_content"
        android:textColor="#000000"
        android:text="This is Activity 1" />
    <EditText
        android:id="@+id/EditText1"
        android:layout_width="match_parent"
        android:layout_height="46dp" />
    <Button android:text="Next"
        android:id="@+id/Button01"
        android:layout_width="300px"
        android:layout_height="200px"
        android:textSize="30px" />
</LinearLayout>
```

Explicit Intent Example (activity_main2.xml)

```
<?xml version="1.0" encoding="utf-8"?>
<LinearLayout
    xmlns:android="http://schemas.android.com/apk/res/android"
    xmlns:tools="http://schemas.android.com/tools"
    android:orientation="vertical"
    android:layout_width="match_parent"
    android:layout_height="match_parent">
    <TextView android:layout_width="fill_parent"
        android:layout_height="wrap_content"
        android:textColor="#000000"
        android:text="This is Activity 2" />
    <TextView
        android:id="@+id/TextView1"
        android:layout_width="match_parent"
        android:layout_height="46dp"
        android:text="Output"/>
    <Button android:text="Next"
        android:id="@+id/Button02"
        android:layout_width="300px"
        android:layout_height="200px"
        android:textSize="30px" />
</LinearLayout>
```

Explicit Intent Example (MainActivity1.kt)

```
package hk.hkucs.explicitintent

import android.content.Intent
import android.os.Bundle
import android.widget.Button
import android.widget.EditText
import androidx.appcompat.app.AppCompatActivity

open class MainActivity : AppCompatActivity() {
    override fun onCreate(savedInstanceState: Bundle?) {
        super.onCreate(savedInstanceState)
        setContentView(R.layout.activity_main)
        val next = findViewById<Button>(R.id.Button01)
        val input = findViewById<EditText>(R.id.EditText1)
        next.setOnClickListener {
            val inputString = input.text.toString()
            val myIntent = Intent(this, MainActivity2::class.java)
            myIntent.putExtra("InputString", inputString)
            startActivity(myIntent)
        }
    }
}
```

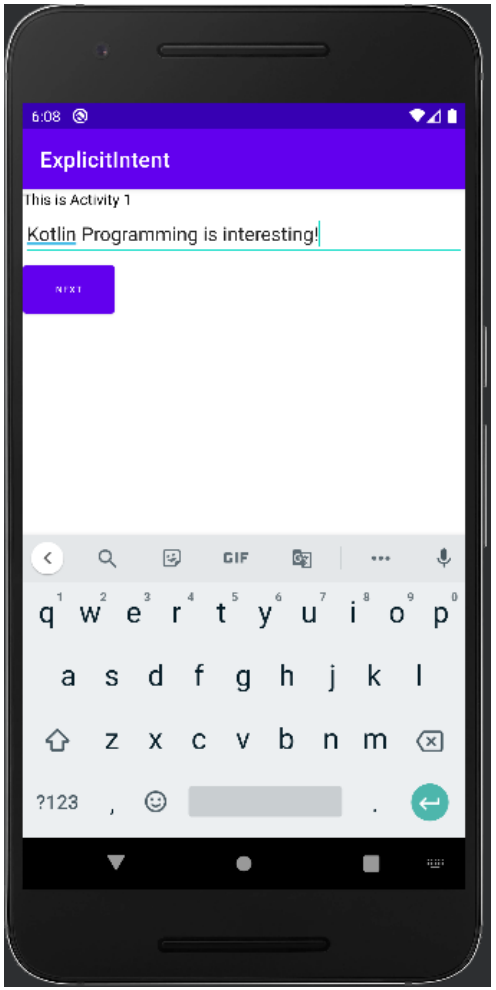
Explicit Intent Example (MainActivity2.kt)

```
package hk.hkucs.explicitintent
import android.content.Intent
import android.os.Bundle
import android.util.Log
import android.widget.Button
import android.widget.TextView
import androidx.appcompat.app.AppCompatActivity

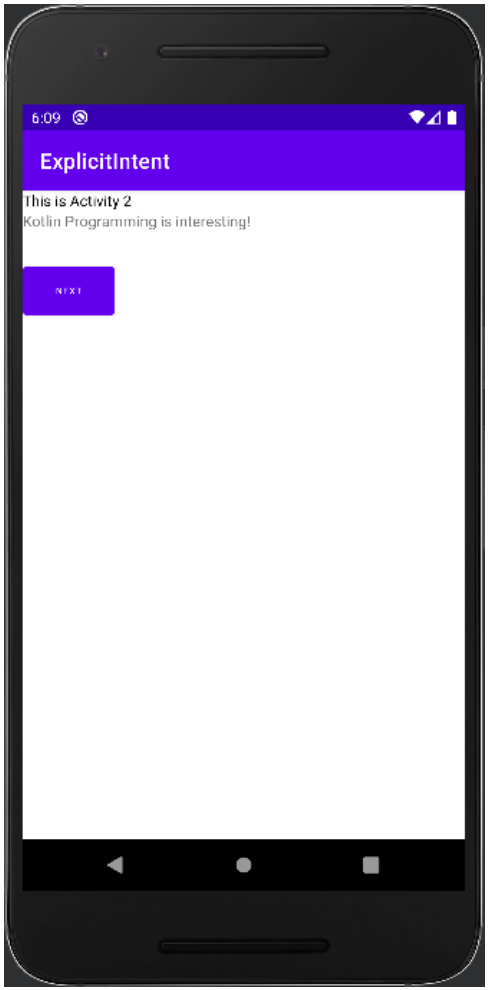
class MainActivity2 : AppCompatActivity() {
    protected override fun onCreate(savedInstanceState: Bundle?) {
        super.onCreate(savedInstanceState)
        setContentView(R.layout.activity_main2)
        val next = findViewById<Button>(R.id.Button02)
        val output = findViewById<TextView>(R.id.TextView1)
        val bundle: Bundle? = intent.extras
        bundle?.let {
            bundle.apply {
                val inputString: String? = getString("InputString")
                output.text = inputString
            }
        }
        next.setOnClickListener {
            val myIntent = Intent(this, MainActivity::class.java)
            startActivity(myIntent)
        }
    }
}
```

Explicit Intent Example

MainActivity1:



MainActivity2:



Android Studio

For complete example, please refer to the project "Explicit Intent".

Passing Extra Data



Sender Activity

- To pass more data from the first activity to the second activity, you may use the following:

```
myIntent = Intent(this, MainActivity2::class.java);
myIntent.putExtra(“ID", 6999)
startActivity(myIntent)
```

Receiver Activity

- In the second activity, you can obtain the value of ID as follows:

```
val bundle: Bundle? = intent.extras
bundle?.let {
        bundle.apply {
                val inputString: String? = getString("ID")   // id = 6999
        }
}
```

- Can use getInt("ID") if ID is an integer
- Can use getLong("ID") if ID is a long integer

Passing Arrays and Objects

You can pass arrays and objects using intents.

```
class MainActivity1 : AppCompatActivity() {
        val intArrayList = arrayListOf(1,2,3)
        val strArrayList = arrayListOf("Alice", "Bob", "Catherine")
        val myCal : Calendar
        …
        ok.setOnClickListener {
                val intent = Intent(this, MainActivity2::class.java)
                intent.putExtra("IntArrayList", intArrayList)
                intent.putExtra("StrArrayList", strArrayList)
                intent.putExtra("Calendar", myCal)
                …
```

Passing Arrays and Objects

You can pass arrays and objects using intents.

```
class MainActivity2 : AppCompatActivity() {
        val bundle: Bundle? = intent.extras
        bundle?.let {
                bundle.apply {
                        val intArrayList: ArrayList<Int>? = getIntArrayList(“IntArrayList")
                        val strArrayList: ArrayList<String>? = getStringArrayList(“StrArrayList")
                        val myCal: Calendar? = getSerializable(“Calendar”)
                }
        }
        …
```

Serialization is the conversion of an object to a series of bytes, so that the object can be easily saved to persistent storage or streamed across a communication link or from one virtual machine to another. The byte stream can then be desterilized - converted into a replica of the original object.



Fragment

What is a Fragment?

- A Fragment represents a behavior or a portion of user interface in a FragmentActivity.
- You can combine multiple fragments in a single activity to build a multi-pane UI and reuse a fragment in multiple activities.
- A fragment is like a modular section of an activity, which has its own lifecycle, receives its own input events, and which you can add or remove while the activity is running (like a "sub-activity" that you can reuse in different activities).

Lifecycle of a Fragment

You should implement at least the following:

onCreate():

- The system calls this when creating the fragment. Within your implementation, you should initialize essential components of the fragment that you want to retain when the fragment is paused or stopped, then resumed.

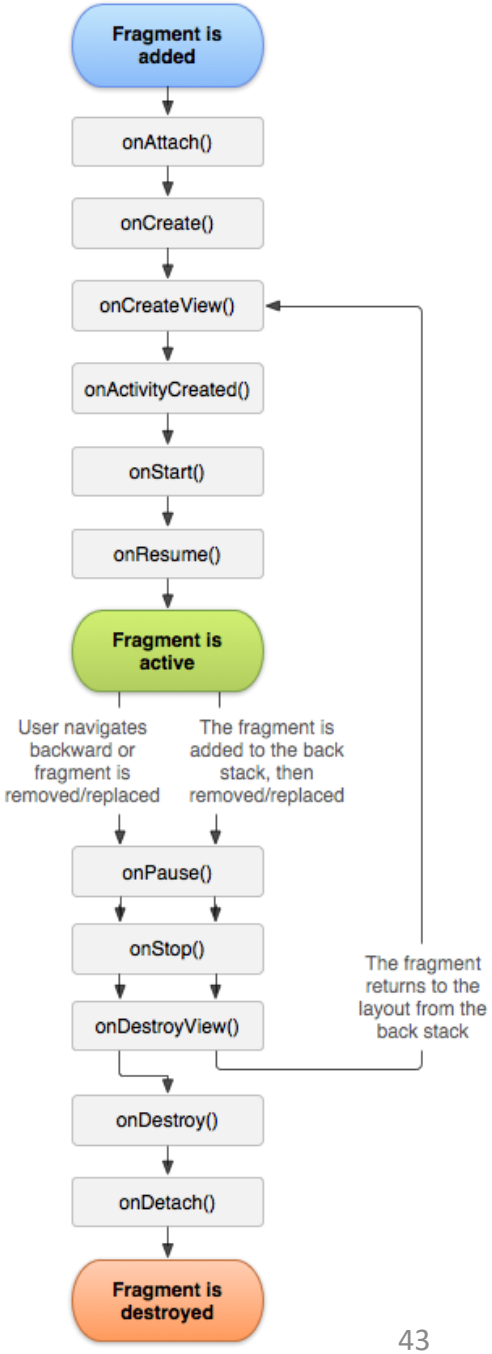
onCreateView()

- The system calls this when it's time for the fragment to draw its user interface for the first time.

You can also implement the following:

onPause()

- The system calls this method as the first indication that the user is leaving the fragment.



Basic Steps

- Modify the layout to include one or more regions for the fragments.
- For each fragment,
  - create a layout.
  - create a class which extents Fragment and implement essential methods.
- In the main activity, at where you want to add the fragment, include the following code:

Java:

```
FragmentManager manager = getSupportFragmentManager();
FragmentTransaction ft = manager.beginTransaction();
ft.replace(<ID of region in main layout>, <class name of fragment>(), <number tag to represent fragment>);
ft.commitAllowingStateLoss();
```

Kotlin:

```
val manager: FragmentManager = supportFragmentManager
val ft: FragmentTransaction = manager.beginTransaction()
if (fragment != null)
    ft.replace(<ID of region in main layout>, <class name of fragment>(), <number tag to represent fragment>);
ft.commitAllowingStateLoss()
```

- At where you want to add a fragment to back stack, call the following:

```
ft.addToBackStack(<number tag representing the fragment>)
```

Example (activity_main.xml)

```
<?xml version="1.0" encoding="utf-8"?>
<LinearLayout xmlns:android="http://schemas.android.com/apk/res/android"
    xmlns:app="http://schemas.android.com/apk/res-auto"
    xmlns:tools="http://schemas.android.com/tools"
    android:layout_width="match_parent"
    android:layout_height="match_parent"
    android:orientation="vertical"
    tools:context=".MainActivity">

    <TextView
        android:layout_width="match_parent"
        android:layout_height="50dp"
        android:layout_margin="10dp"
        android:id="@+id/fr1"
        android:gravity="center"
        android:textSize="20dp"
        android:textColor="#fff"
        android:background="#e11010"
        android:textStyle="bold"
        android:text="Fragment 1" />

    <TextView
        android:layout_width="match_parent"
        android:layout_height="50dp"
        android:id="@+id/fr2"
        android:layout_margin="10dp"
        android:gravity="center"
        android:textSize="20dp"
        android:textColor="#fff"
        android:background="#1010e1"
        android:textStyle="bold"
        android:text="Fragment 2"/>

    <FrameLayout
        android:layout_width="match_parent"
        android:layout_height="match_parent"
        android:id="@+id/frame_container"
        android:layout_margin="15dp">

    </FrameLayout>

</LinearLayout>
```



Example (fragment_fragment_one.xml)

```
<?xml version="1.0" encoding="utf-8"?>
<FrameLayout xmlns:android="http://schemas.android.com/apk/res/android"
    xmlns:tools="http://schemas.android.com/tools"
    android:layout_width="match_parent"
    android:layout_height="match_parent"
    tools:context=".FragmentOne">

    <TextView
        android:layout_width="match_parent"
        android:layout_height="match_parent"
        android:textStyle="bold"
        android:textSize="30sp"
        android:gravity="center"
        android:textColor="#fff"
        android:background="#e70ba5"
        android:text="I am Fragment 1" />
</FrameLayout>
```



Example (FragmentOne.kt)

```
package hk.hkucs.fragment

import android.os.Bundle
import android.view.LayoutInflater
import android.view.View
import android.view.ViewGroup
import androidx.fragment.app.Fragment

class FragmentOne : Fragment() {
    override fun onCreate(savedInstanceState: Bundle?) {
        super.onCreate(savedInstanceState)
    }

    override fun onCreateView(
        inflater: LayoutInflater,
        container: ViewGroup?,
        savedInstanceState: Bundle?
    ): View {
        // Inflate the layout for this fragment
        return inflater.inflate(R.layout.fragment_fragment_one, container, false)
    }
}
```

Example (fragment_fragment_two.xml)

```
<?xml version="1.0" encoding="utf-8"?>
<FrameLayout xmlns:android="http://schemas.android.com/apk/res/android"
    xmlns:tools="http://schemas.android.com/tools"
    android:layout_width="match_parent"
    android:layout_height="match_parent"
    tools:context=".FragmentTwo">

    <TextView
        android:layout_width="match_parent"
        android:layout_height="match_parent"
        android:textStyle="bold"
        android:textSize="30sp"
        android:gravity="center"
        android:textColor="#fff"
        android:background="#e7880b"
        android:text="I am Fragment 2" />
</FrameLayout>
```



Example (FragmentTwo.kt)

```
package hk.hkucs.fragment

import android.os.Bundle
import android.view.LayoutInflater
import android.view.View
import android.view.ViewGroup
import androidx.fragment.app.Fragment

class FragmentTwo : Fragment() {
    override fun onCreate(savedInstanceState: Bundle?) {
        super.onCreate(savedInstanceState)
    }

    override fun onCreateView(
        inflater: LayoutInflater, container: ViewGroup?,
        savedInstanceState: Bundle?
    ): View {
        // Inflate the layout for this fragment
        return inflater.inflate(R.layout.fragment_fragment_two, container, false)
    }
}
```

Example (MainActivity.java)

```
package hk.hkucs.fragment

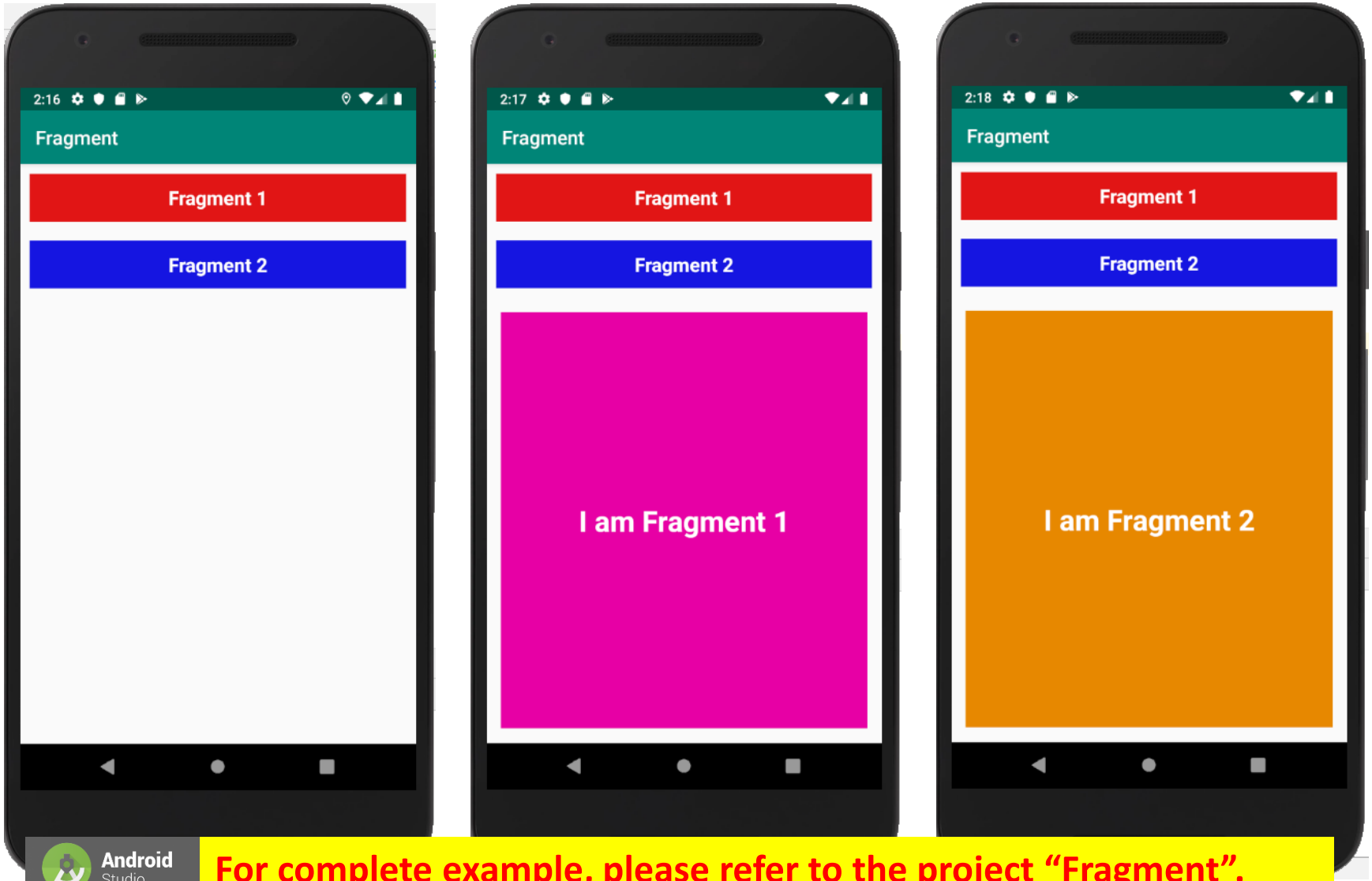
import android.os.Bundle
import android.view.View
import android.widget.TextView
import androidx.appcompat.app.AppCompatActivity
import androidx.fragment.app.Fragment
import androidx.fragment.app.FragmentManager
import androidx.fragment.app.FragmentTransaction

class MainActivity : AppCompatActivity() {
    override fun onCreate(savedInstanceState: Bundle?) {
        super.onCreate(savedInstanceState)
        setContentView(R.layout.activity_main)
        val tv1 = findViewById<TextView>(R.id.fr1)
        val tv2 = findViewById<TextView>(R.id.fr2)
        tv1.setOnClickListener(View.OnClickListener { addFragment(FragmentOne(), false, "one") })
        tv2.setOnClickListener(View.OnClickListener { addFragment(FragmentTwo(), false, "two") })
    }
```

Example (MainActivity.java)

```
    fun addFragment(fragment: Fragment?, addToBackStack: Boolean, tag: String?) {
        val manager: FragmentManager = supportFragmentManager
        val ft: FragmentTransaction = manager.beginTransaction()
        if (addToBackStack) {
            ft.addToBackStack(tag)
        }
        if (fragment != null) {
            ft.replace(R.id.frame_container, fragment, tag)
        }
        ft.commitAllowingStateLoss()
    }
}
```

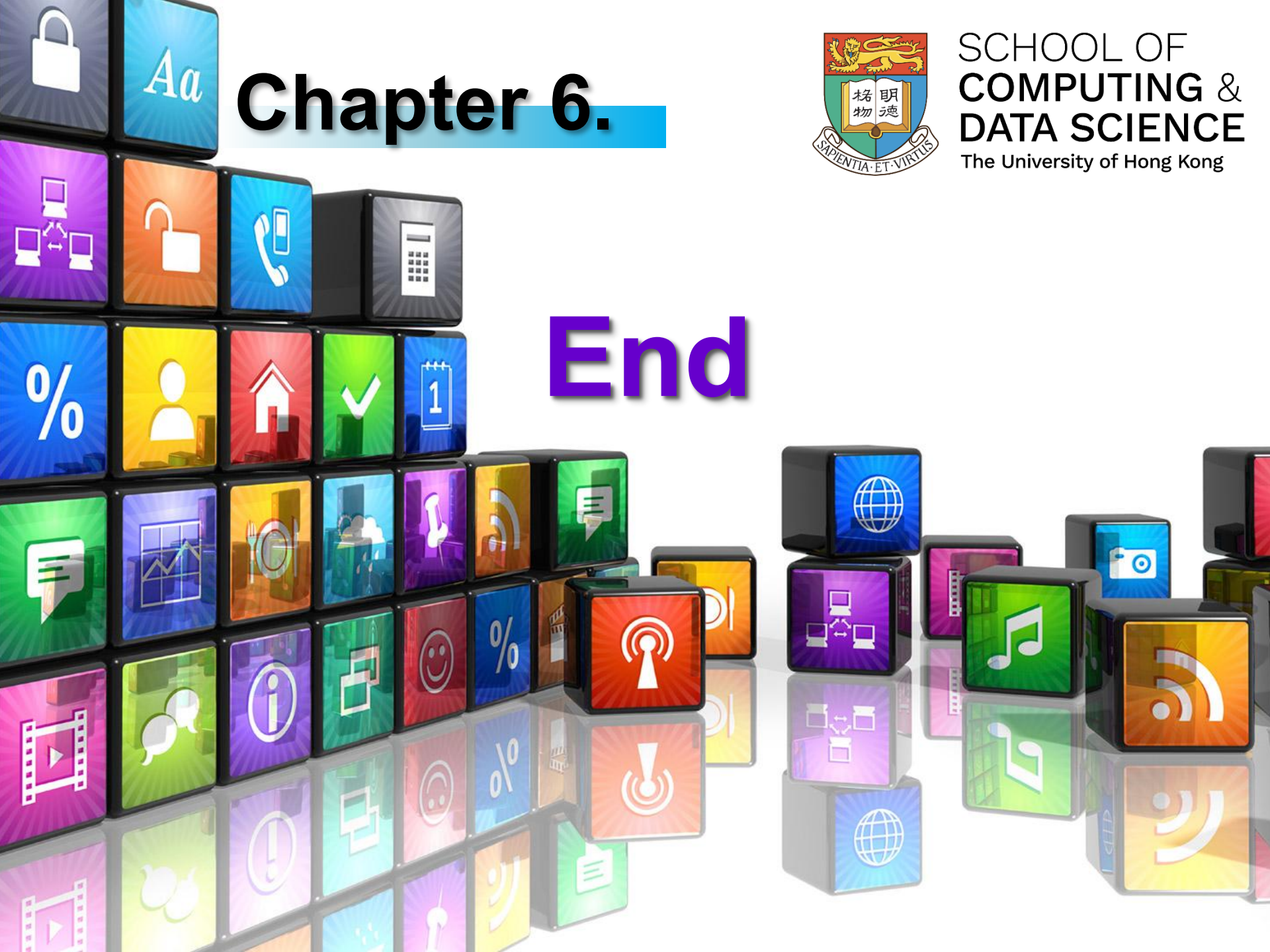
Example



Android Studio

For complete example, please refer to the project "Fragment".

Chapter 6.



SCHOOL OF COMPUTING & DATA SCIENCE
The University of Hong Kong

End